



Design Document

Gerenciamento de Restaurantes

Luiz Henrique, Edvaldo, José Wallas, Pedro Bruno

04/2024

Sumário

1. Introdução.....	4
2. Visão Geral do Produto.....	4
3. Análise de Requisitos (Prioridade: 1-Alta,2-Média e 3-Baixa).....	5
• Requisitos Funcionais.....	5
O sistema deve permitir que:.....	5
• Requisitos Não Funcionais.....	7
4. Arquitetura Do Sistema.....	8
• Diagrama de Casos de Uso.....	9
• Diagrama de Classes.....	10
• Diagrama de Nível de Acesso.....	12
• Diagrama de Camadas.....	13
• Modelo de dados.....	13
○ Tabelas Principais.....	13
○ Relacionamentos Principais.....	18
• Integrações.....	19
5. Interface de Usuário.....	20
• Visão Geral.....	20
• Telas.....	21
• Fluxos de navegação.....	23
6. Plano de Implementação.....	24
• Roadmap de desenvolvimento.....	24
• Tecnologias e Ferramentas.....	25
• Decisões de design.....	25
• Prioridades e entregas parciais.....	26
• Equipe responsável e papéis.....	26
7. Testes.....	28
8. Planejamento do MVP.....	29
• Introdução.....	29
• Funcionalidades Essenciais.....	30
• Hipóteses para Validar.....	30
• Fluxo Principal (Pedidos).....	31
• Riscos a serem analisados.....	31
• Plano de Validação.....	31
9. Boas Práticas de Código na Aplicação (Task 02).....	32
• Single Responsibility Principle (SRP) – Princípio da Responsabilidade Única.....	32
• Open/Closed Principle (OCP) – Princípio Aberto/Fechado.....	33
• Liskov Substitution Principle (LSP) – Princípio de Substituição de Liskov.....	35
• Interface Segregation Principle (ISP) – Princípio da Segregação de Interfaces.....	36

10. Conclusão.....	37
---------------------------	-----------

Glossário

Administrador: Usuário com acesso total ao sistema (Cadastra restaurantes e usuários)

Cliente: Usuário que realiza pedidos no restaurante (local ou delivery) através do sistema.

Gerente: Responsável por gerenciar um restaurante específico (cardápios, pedidos, mesas, pedidos e etc)

1. Introdução

Este documento visa apresentar uma solução digital para o gerenciamento de pedidos em restaurantes para os gerentes/donos dos mesmos, descrevendo a arquitetura do sistema, suas funcionalidades e requisitos.

2. Visão Geral do Produto

- O que é?
 - Um sistema focado em restaurantes que:
 - Substitui anotações manuais de pedidos por um fluxo digital.
 - Permite que os clientes façam pedidos no estabelecimento ou para entrega, através de algum dispositivo digital.
- Por que existe?
 - Problema: Cardápios e pedidos físicos gastam tempo com anotações, sobrecarregando os funcionários, o que acarreta em mais tempo de espera para servir ou entregar os pedidos
 - Diferencial: Simples, intuitivo e prático (menos burocracia para os clientes e para os funcionários do restaurante).
- Público Alvo
 - **Gerente/Dono:** Indivíduo que possui ou gerencia um determinado restaurante e está em busca de inserir a tecnologia em seu estabelecimento para melhorar o atendimento ao cliente e a performance de seus funcionários.

- **Cliente:** Indivíduo que está a procura de um estabelecimento que forneça uma boa refeição e um atendimento de qualidade,prático e eficaz.
- Quais são as funcionalidades?
 - Cadastro e gerenciamento de usuários
 - Cadastro e gerenciamento de restaurantes
 - Cadastro e gerenciamento de clientes, mesas, cardápios,categorias, pratos,pagamentos e entregas do restaurante
 - Exibição e filtragem de restaurantes por categoria e endereço
 - Exibição e filtragem de cardápios por categoria
 - Realização de pedidos locais ou delivery
 - Inserção de pratos em cardápios

3. Análise de Requisitos (Prioridade: 1-Alta,2-Média e 3-Baixa)

- Requisitos Funcionais

O sistema deve permitir que:

RF01: O Administrador cadastre restaurantes (P1)

RF02: O Administrador gerencie os restaurantes (P1)

RF03: O Cliente busque restaurantes por endereço (P3)

RF04: O Cliente busque restaurantes por categoria (P2)

RF05: O Gerente cadastre cardápios (P1)

RF06: O Gerente gerencie os cardápios (P1)

RF07: O Cliente busque cardápios por categoria (P2)

RF08: O Gerente cadastre categorias (P1)

RF09: O Gerente gerencie categorias (P1)

RF10: O Gerente cadastre pratos (P1)

RF11: O Gerente gerencie pratos (P1)

RF12: O Gerente insira pratos nos cardápios (P1)

RF13: O Cliente busque pratos por cardapio (P1)

RF14: O Gerente cadastre mesas (P1)

RF15: O Gerente gerencie as mesas (P1)

RF16: O Cliente busque mesas ocupadas (P2)

RF17: O Cliente busque mesas desocupadas (P2)

RF18: O Gerente cadastre endereços (P1)

RF19: O Gerente gerencie endereços (P1)

RF20: O Gerente cadastre pedidos (P1)

RF21: O Gerente gerencie pedidos (P1)

RF22: O Cliente insira pratos no pedido (P1)

RF23: O Administrador cadastre usuários (P1)

RF24: O Administrador gerencie usuários (P1)

RF25: O Cliente realize seu cadastro (P2)

RF26: O Gerente cadastre entregas (P3)

RF27: O Gerente gerencie entregas (P3)

RF28: O Gerente cadastre pagamentos (P1)

RF29: O Gerente gerencie pagamentos (P1)

RF30: O Cliente realize pedido (P1)

RF31: O Administrador redefine senhas de usuários (P1)

RF32: Seja enviado um e-mail de confirmação após cadastro de cliente (P2)

RF33: Seja realizado o bloqueio temporário de usuários após X tentativas falhas de login (P2)

RF34: Seja realizado o upload de imagens para pratos/cardápios (P2)

RF35: Seja realizada a integração com gateways de pagamento (ex: Mercado Pago, PagSeguro) (P2)

RF36: O sistema deve exportar dados de vendas em CSV/PDF (P3)

- **Requisitos Não Funcionais**

RNF01: O sistema deve ter tempo de resposta inferior a 3s para 95% das requisições em casos normais de uso (P2)

RNF02: Dados sensíveis (senhas, CPF) devem ser criptografados (P1)

RNF03: O sistema deve suportar 50 usuários concorrentes sem degradação de performance (P2)

RNF04: O sistema deve bloquear contas após 5 tentativas falhas de login por 10 minutos (P1)

RNF05: Telas críticas (como pedidos) devem ter feedback visual claro (ex: animação de "sucesso") (P2)

RNF06: Backup automático diário dos dados com retenção de 7 dias (P1)

RNF07: O sistema deve ter 99% de disponibilidade (P1)

RNF08: O sistema deve funcionar em navegadores modernos como (Chrome, Firefox, Edge, Safari) nas 3 versões mais recentes (P2)

RNF09: Suporte a iOS 14+ e Android 10+ para versão mobile (P2)

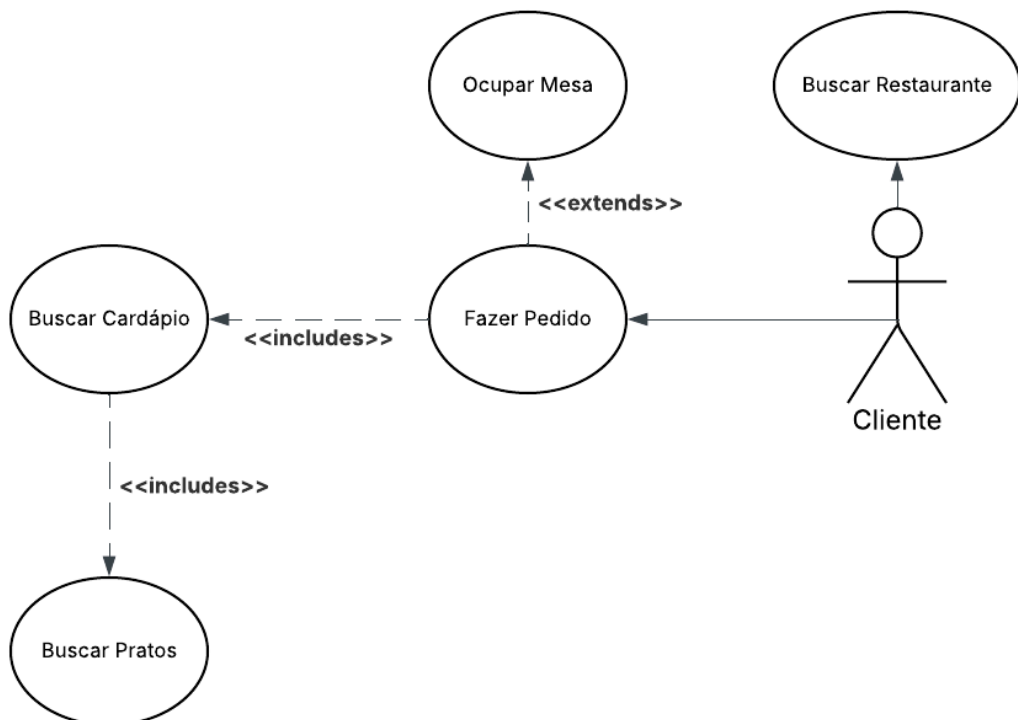
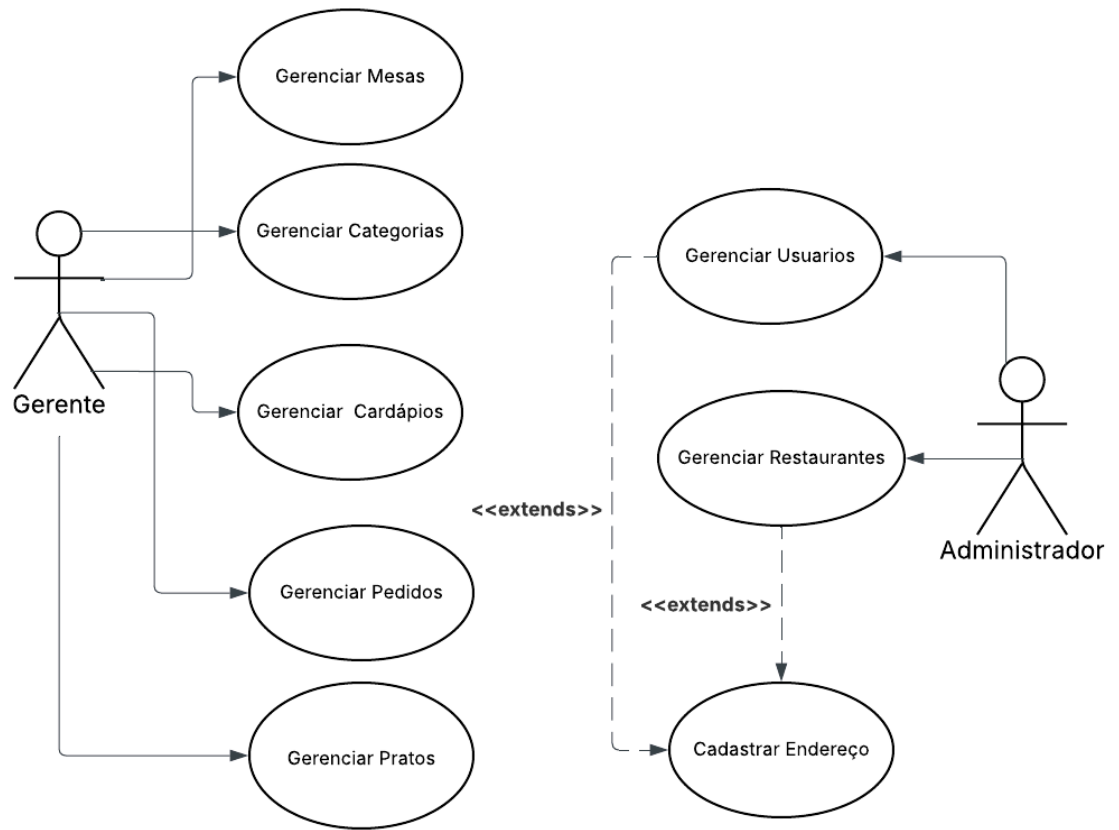
RNF10: A arquitetura deve permitir escalar para 10x o volume atual sem refatoração (P2)

RNF11: Banco de dados deve suportar até 12.000 registros de pedidos/mês (P2)

4. Arquitetura Do Sistema

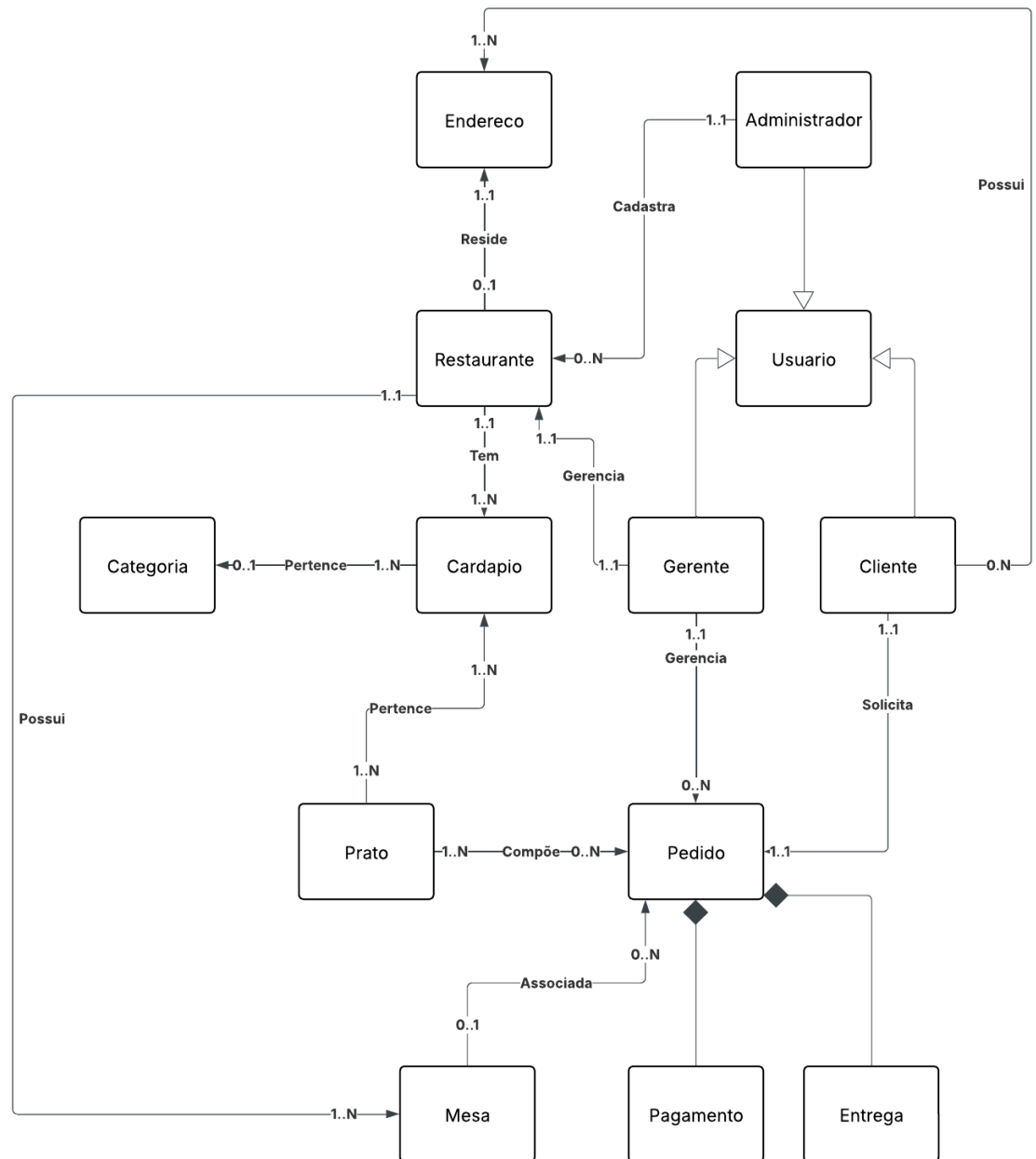
O sistema, pensado para ser desenvolvido inicialmente (MVP) em Arquitetura Monolítica. A justificativa da escolha envolve o tamanho da equipe e seu nível de especialização, a disposição de recursos limitada, a simplicidade e praticidade necessária para o desenvolvimento do MVP.

- Diagrama de Casos de Uso



- Diagrama de Classes

Diagrama de Classes



Usuario
email: String # nome: String # senha: String # cpf: String

Pedido
- status: String - descricao: String - data: Datetime - total: Money
Prato
- nome: String - ingredientes: String - descricao: String - valor: Money

Restaurante
- nome: String - telefone: String - descricao: String - horario_abertura: Time - horario_fechamento: Time

Endereco
- cidade: String - estado:String - bairro: String - logradouro: String - numero: Int - complemento: String - cep: String

Entrega
- descricao:String - status: String - tempo_estimado: String

Administrador
- codigo: String

Gerente
- matricula: String

Cliente
- telefone: String

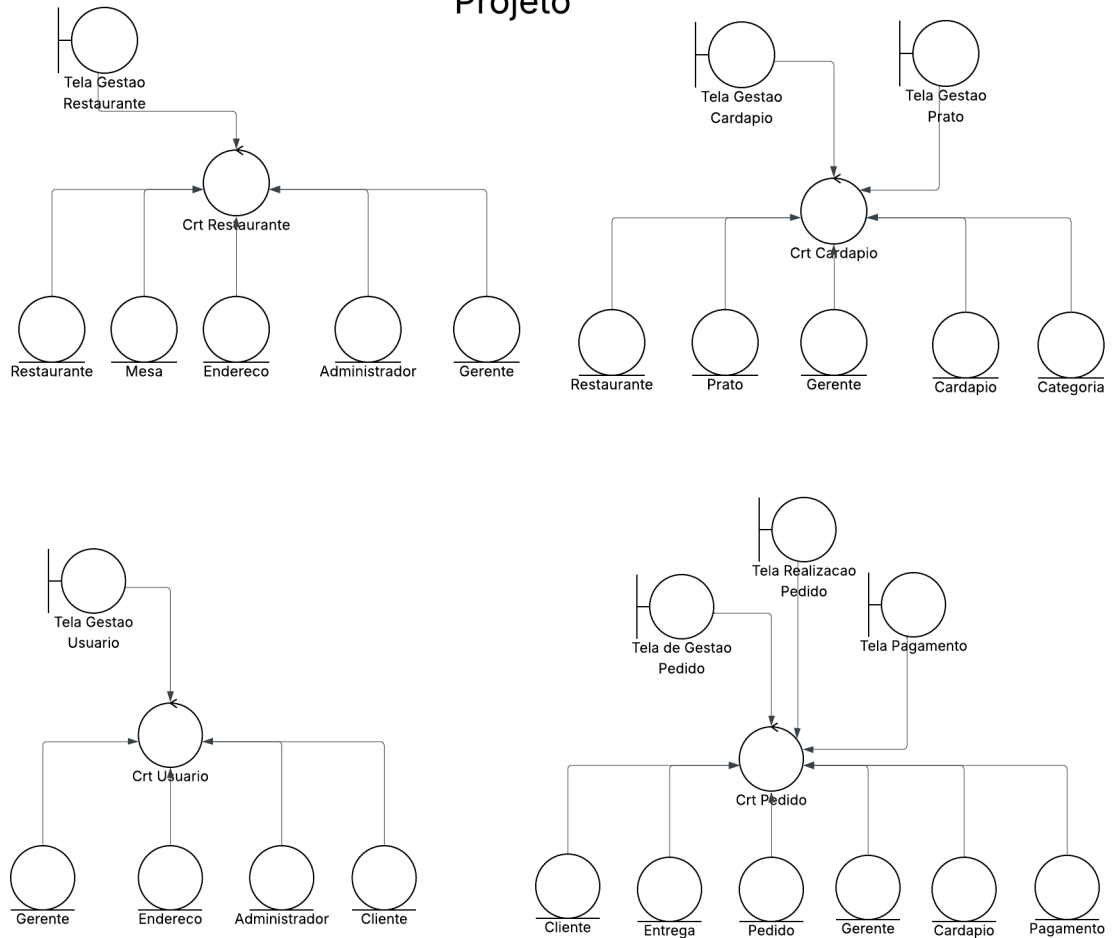
Mesa
- numero: Int - ocupada: Boolean - capacidade: Int

Cardapio
- tipo: String - valor_total: Money - descrição: String

Categoria
- nome: String

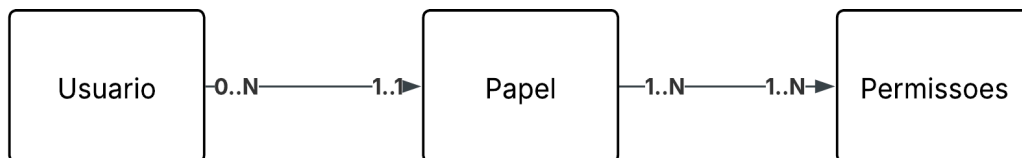
Pagamento
- forma: String - valor: Money

Diagrama de Classes em Nivel de Projeto

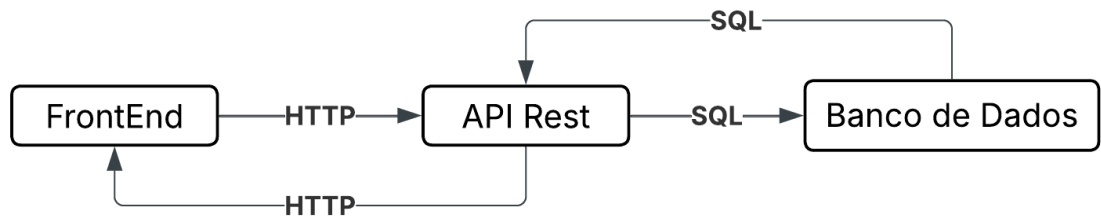


● Diagrama de Nível de Acesso

A nível de armazenamento dos usuários, o sistema contará com um modelo de *Role Has Permissions*, que consiste no usuário possuir um papel (*role*), que por sua vez possui permissões (*permissions*) para gerenciar o nível de acesso ao sistema.



- Diagrama de Camadas



- Modelo de dados

- Tabelas Principais

- **Usuários e Autenticação**

- 1. User

- id (PK)
- email (String)
- name (String)
- key (String)
- cpf (String)
- phone (String)
- active (Boolean)
- created_at (DateTime)
- updated_at (DateTime)

- 2. Role

- id (PK)
- name (String)
- description (String)
- created_at (DateTime)

- updated_at (DateTime)

3. Permission

- id (PK)
- name (String)
- description (String)
- code (String)
- created_at (DateTime)
- updated_at (DateTime)

4. User_Role

- user_id (FK)
- role_id (FK)

5. Role_Permission

- role_id (FK)
- permission_id (FK)

■ Tipos Específicos de Usuários

6. Client

- user_id (PK, FK)
- phone (String)

7. Manager

- user_id (PK, FK)
- registration (String)

8. Administrator

- user_id (PK, FK)

- code (String)

■ Restaurante e Operações

9. Restaurant

- id (PK)
- name (String)
- phone (String)
- opening_time (Time)
- closing_time (Time)
- active (Boolean)
- created_at (DateTime)
- updated_at (DateTime)

10. Address

- id (PK)
- restaurant_id (FK)
- city (String)
- state (String)
- neighborhood (String)
- street (String)
- number (Int)
- complement (String)
- zip_code (String)

11. Table

- id (PK)

- restaurant_id (FK)
- number (Int)
- occupied (Boolean)
- capacity (Int)

■ Cardápio e Produtos

12. Menu

- id (PK)
- restaurant_id (FK)
- type (String)
- total_value (Decimal)
- description (String)
- active (Boolean)
- created_at (DateTime)
- updated_at (DateTime)

13. Category

- id (PK)
- name (String)
- description (String)
- menu_id (FK)

14. Dish

- id (PK)
- category_id (FK)
- name (String)

- description (String)
- value (Decimal)
- active (Boolean)
- image_url (String)
- created_at (DateTime)
- updated_at (DateTime)

■ Pedidos e Pagamentos

15. Order

- id (PK)
- client_id (FK)
- table_id (FK, nullable)
- status (String)
- date (DateTime)
- total (Decimal)
- observations (String)
- created_at (DateTime)
- updated_at (DateTime)

16. OrderItem

- id (PK)
- order_id (FK)
- dish_id (FK)
- quantity (Int)
- unity_value (Decimal)

- observations (String)

17. Delivery

- id (PK)
- order_id (FK)
- address_id (FK)
- description (String)
- status (String)
- estimated_time (Int)
- created_at (DateTime)
- updated_at (DateTime)

18. Payment

- id (PK)
- order_id (FK)
- method (String)
- value (Decimal)
- status (String)
- date (DateTime)
- transaction_code (String)

○ Relacionamentos Principais

1. Usuário pode ter vários Papéis (M:N) via Usuario_Papel
2. Papel pode ter várias Permissões (M:N) via Papel_Permissao
3. Cliente, Gerente e Administrador são especializações de Usuário (herança)

4. Restaurante tem um Endereço (1:1)
5. Restaurante tem várias Mesas (1:N)
6. Restaurante tem vários Cardápios (1:N)
7. Cardápio tem várias Categorias (1:N)
8. Categoria tem vários Pratos (1:N)
9. Cliente faz vários Pedidos (1:N)
10. Pedido pode ser associado a uma Mesa (1:1, opcional)
11. Pedido contém vários ItensPedido (1:N)
12. Pedido pode ter uma Entrega (1:1, opcional)
13. Pedido tem um Pagamento (1:1)

- Integrações

- APIs/Serviços externos

- Serviço de E-mail (RF32):

- SendGrid para envio de e-mails de confirmação de cadastro/pedidos.
 - Justificativa: Requisito para validação de contas (RF32).

- Gateways de Pagamento (RF35):

- Integração com Mercado Pago ou PagSeguro para processar pagamentos digitais (cartão/PIX).
 - Justificativa: Necessário para fechar o ciclo de pedidos (RF28/RF29).

5. Interface de Usuário

- Visão Geral

- Objetivos:

- **Intuitiva** : Garantir que o software seja fácil de entender desde o primeiro uso, minimizando a necessidade de treinamento. Isso inclui:

- **Padrões visuais familiares**: Botões de ação destacados (ex.: "Finalizar Pedido" em verde vibrante).
 - **Ícones universais**: Como um carrinho para pedidos ou uma lupa para buscar pratos, evitando textos longos.

- **Acessível**: Garantir que o software seja intuitivo e fácil de usar, mesmo para funcionários com pouca experiência digital. Isso inclui:

- **Interface clara**: Menos termos técnicos, ícones autoexplicativos e fluxos lógicos (ex.: fazer um pedido em poucos cliques).
 - **Feedback visual/sonoro**: Confirmações de ações (ex.: "Pedido enviado!" ou um beep ao registrar um pagamento).

- **Responsiva** : Garantir que a interface se adapte automaticamente a diferentes tamanhos de telas e dispositivos (como smartphones, tablets e desktops), garantindo usabilidade e visual otimizados em qualquer resolução. Isso envolve ajustes fluidos de layout, imagens, textos e elementos interativos.

- Plataformas Suportadas

- Web, Mobile e Desktop.

- Telas

1. Telas de Autenticação e Acesso

- Login
 - Campos: E-mail, senha, opção "Esqueci minha senha".
 - Botões: "Entrar", "Cadastrar-se" (para clientes).
- Cadastro de Cliente
 - Campos: Nome, e-mail, CPF, telefone, senha.
 - Botão: "Criar conta".

2. Módulo Administrativo (Administrador)

- Dashboard Administrativo
 - Resumo de restaurantes, usuários, métricas.
- Gerenciamento de Usuários
 - Lista de usuários (filtros por tipo: Cliente, Gerente, Admin).
 - CRUD: Adicionar/editar/excluir usuários.
- Gerenciamento de Restaurantes
 - Lista de restaurantes (com status "ativo/inativo").
 - CRUD: Cadastrar/editar/desativar restaurantes.

3. Módulo de Restaurante (Gerente)

- Dashboard do Gerente
 - Visão geral: Pedidos ativos, mesas ocupadas, entregas pendentes.
- Gerenciamento de Cardápios

- Lista de cardápios (ex: "Bebidas", "Pratos Principais").
- CRUD: Criar/editar/remover cardápios.
- Gerenciamento de Pratos
 - Lista de pratos (com filtros por categoria).
 - Campos: Nome, descrição, valor, imagem, status (ativo/inativo).
- Gerenciamento de Mesas
 - Mapa visual das mesas (ocupadas/disponíveis).
 - Opção: Marcar mesa como ocupada/livre.

4. Módulo de Pedidos (Cliente e Gerente)

- Busca de Restaurantes (Cliente)
 - Filtros: Por endereço, categoria (ex: "Pizzaria", "Japonesa").
 - Lista com cards de restaurantes (imagem, nome, avaliação).
- Visualização de Cardápio (Cliente)
 - Categorias (ex: "Entradas", "Sobremesas").
 - Lista de pratos com opção de adicionar ao pedido.
- Carrinho de Pedidos (Cliente)
 - Resumo dos itens selecionados, valor total.
 - Opções: "Fazer pedido local" (mesa) ou "Delivery" (endereço).

- Acompanhamento de Pedidos (Cliente/Gerente)
 - Status: "Em preparo", "Pronto", "Entregue".
 - Botões: (Gerente) "Marcar como concluído".

5. Pagamento e Entrega

- Seleção de Pagamento (Cliente)
 - Opções: Cartão, PIX, dinheiro.
 - Campos: Dados do cartão (se aplicável).
- Acompanhamento de Entrega (Cliente/Gerente)
 - Mapa com rota (para delivery).
 - Tempo estimado, status ("Saiu para entrega").

6. Telas de Relatórios (Gerente/Administrador)

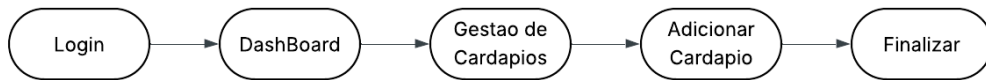
- Relatório de Vendas
 - Vendas por período, pratos mais vendidos.
- Relatório de Ocupação
 - Horários de pico, tempo médio por mesa.

● Fluxos de navegação

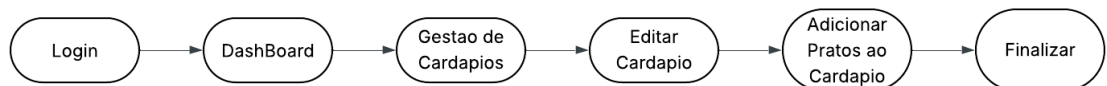
Administrador: Cadastro de Restaurante



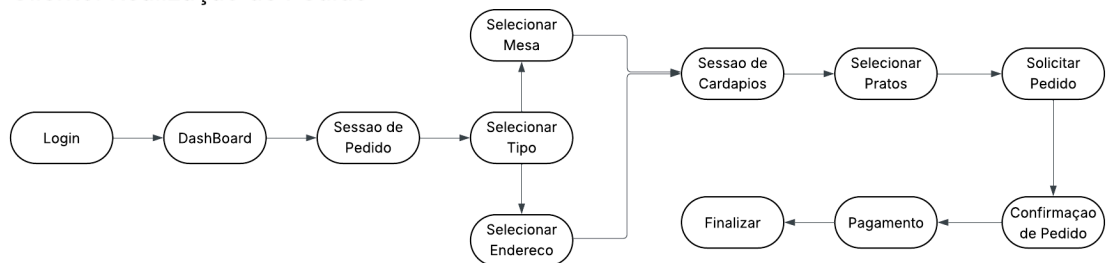
Gerente: Cadastro de Cardápio



Gerente: Inserção de Pratos no Cardápio



Cliente: Realização de Pedido



6. Plano de Implementação

- Roadmap de desenvolvimento

- Fase 1: Núcleo Administrativo (1 - 2 Meses)

- Design das Telas
 - Criação do banco de dados
 - Módulos de usuários (Administrador, Gerente e Cliente) completo com CRUD, rotas, permissões, autenticação e etc
 - Criação do CRUD de Endereço
 - Telas de Login e dos Usuários

- Fase 2: Núcleo de Restaurante (2 - 4 Semanas)

- Criação dos CRUDs de Restaurante, Cardápio e Prato

- Criação do CRUD de Categoria
 - Criação das telas de Restaurante, Cardápio e Prato
- Fase 3: Núcleo de Pedido (2 - 4 Semanas)
 - Criação dos CRUDs de Pedido, Mesa, Pagamento e Entrega
 - Criação das telas de Pedido, Entrega e Pagamento
- Fase 4: Adições e Integrações externas (2 - 4 Semanas)
 - Adição de Autocadastro de Clientes
 - Adição de Formas de Pagamento
- Tecnologias e Ferramentas
 - Design: Figma
 - Banco de Dados: Postgresql
 - BackEnd: Java/Spring Boot
 - FrontEnd: Next.Js, React e Typescript
 - Testes: Postman e JUnit
 - Versionamento de Código: Git e GitHub
- Decisões de design
 - Será utilizado o padrão Rest pela simplicidade, facilidade de implementação e compatibilidade com diversas plataformas
 - A principal justificativa da escolha das tecnologias é a familiaridade dos membros com as mesmas, além de visar a simplicidade e facilidade em desenvolver tanto o MVP quanto o produto completo.
 - Em relação ao banco de dados escolhido, o modelo relacional é o mais viável por conta das diversas relações entre as entidades/classes.

- Prioridades e entregas parciais
 - A prioridade é a entrega do MVP (*Minimun Viable Product*), que será detalhado mais abaixo junto com suas entregas parciais.
- Equipe responsável e papéis
 - Edvaldo
 - **Sobre:**
 - **Formação:** Graduando em Engenharia de Computação
 - **Conhecimentos e Habilidades:**
 - **FrontEnd:** HTML,CSS,TypeScript,Next.js e React
 - **BackEnd:** C,Python,Java e Spring Boot
 - **Banco de Dados:** Postgresql e Mysql
 - **Versionamento de Código:** Git e GitHub
 - **Funções:**
 - Criação dos Protótipos das Telas
 - Criação do Banco de Dados
 - Desenvolvimento do BackEnd
 - Desenvolvimento do FrontEnd
 - Testes
 - José Wallas
 - **Sobre:** Graduando em Engenharia de Computação
 - **Funções:**
 - Criação dos Protótipos das Telas

- Criação do Banco de Dados
- Desenvolvimento do BackEnd
- Desenvolvimento do FrontEnd
- Testes

■ Luiz Henrique

● Sobre:

- **Formação:** Graduando em Engenharia de Computação, Técnico em Informática

○ Conhecimentos e Habilidades:

- **FrontEnd:** Next.js, React e Vue
- **BackEnd:** Node.js, Nest
- **Banco de Dados:** Postgresql, Mysql, MongoDB
- **Versionamento de Código:** Git e GitHub

● Funções:

- Criação dos Protótipos das Telas
- Criação do Banco de Dados
- Desenvolvimento do BackEnd
- Desenvolvimento do FrontEnd
- Testes

■ Pedro Bruno

- **Sobre:** Graduando em Engenharia de Computação

● Funções:

- Criação dos Protótipos das Telas
- Criação do Banco de Dados

- Desenvolvimento do BackEnd
- Desenvolvimento do FrontEnd
- Testes

7. Testes

- Unitários
 - Objetivo: Validar cada componente isoladamente (classes, funções e etc).
 - Exemplos:
 1. Validação do cálculo do total de um pedido.
 2. Teste de autenticação de usuário (sucesso/falha).
- Integração
 - Objetivo: Verificar a comunicação entre módulos (ex.: API + banco de dados).
 - Exemplos:
 1. Criar um pedido e verificar se ele aparece no dashboard do gerente.
 2. Integração com gateway de pagamento (simulado em ambiente sandbox).
- Usabilidade

- Objetivo: Avaliar a experiência do usuário (clientes, gerentes).
- Exemplos:
 1. Tempo para registrar um pedido pela primeira vez.
 2. Feedback visual claro ao enviar um pedido (ex.: animação de confirmação).
- Métodos: Observação direta, questionários (ex.: escala de 1 a 5).

8. Planejamento do MVP

● Introdução

○ Problema

- O produto facilita a estruturação de pedidos por parte dos garçons e gerentes de um restaurante, evitando problemas comuns com clientes e desorganização da cozinha em relação a pedidos. Além de retirar a necessidade de usar comandas de papel.

○ Público-Alvo

- Gerente: Responsável por lidar com as funções de um determinado restaurante, gerenciando mesas, cardápios, pratos, pedidos e tudo que está incluso nas funcionalidades disponíveis para esse usuário.
- Cliente: Responsável por realizar pedidos, o que inclui escolha de cardápio, seleção de pratos, pagamento do pedido e outras funcionalidades disponíveis para esse usuário

○ Hipóteses a Validar

- Dificuldade de lidar com pedidos por parte de garçons;

- Redução de erros da cozinha do restaurante;

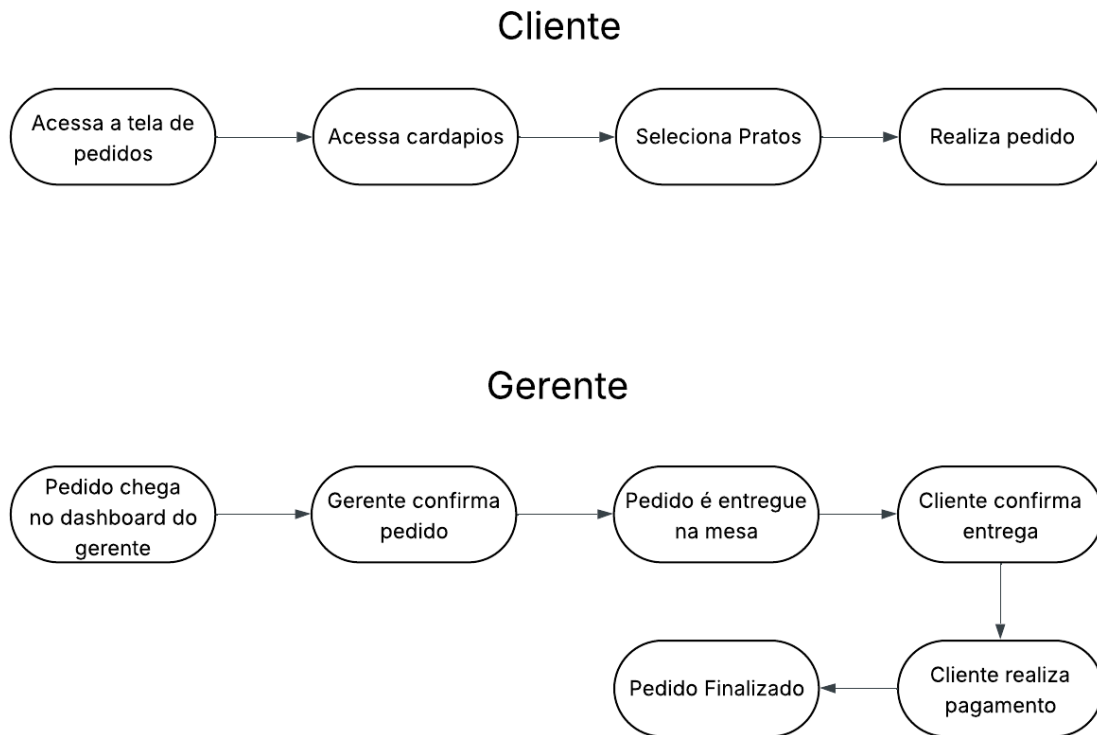
- Funcionalidades Essenciais

- Cadastro e gerenciamento de restaurantes (Administrador)
- Cadastro e Gerenciamento de usuários (Administrador)
- Cadastro e gerenciamento de mesas, cardápios, categorias, pratos e pagamentos do restaurante (Gerente);
- Realização de pedidos locais (Cliente);
- Gerenciamento dos pedidos (Gerente);

- Hipóteses para Validar

- Garçons e gerentes têm dificuldades com pedidos manuais, envolvendo erro, demora e desorganização:
 - Realizar entrevistas com garçons/gerentes e observar o fluxo atual de pedidos em restaurantes.
- Digitalização dos pedidos reduz consideravelmente erros na cozinha:
 - Realizar teste comparativo com antes e depois em um restaurante piloto.
- Clientes preferem fazer pedidos via aplicação em vez de comanda de papel:
 - Realizar teste do protótipo da aplicação com clientes reais para coleta de dados de satisfação.
- Gerentes/donos de estabelecimentos estão dispostos a pagar por uma solução que otimize o fluxo de pedidos:
 - Realizar pesquisa e pré-vendas.

- Fluxo Principal (Pedidos)



- Riscos a serem analisados

- O que ocorrerá se a internet estiver instável?
- Custo x benefício: O restaurante terá ROI (Retorno sobre Investimento) rápido?
- Se houver um grande número de pedidos simultâneos?

- Plano de Validação

- Fase 1: Testes Internos
 - Duração: 2 semanas.
 - Ações:
 1. Testes unitários e de integração cobrindo 80% do código.

2. Simulação de fluxos críticos (ex.: pedido + pagamento).
- Fase 2: Teste Piloto em Restaurantes
 - Duração: 1 mês (pequeno/médio porte) + 1 mês (grande porte).
 - Métricas:
 1. Taxa de erros: Comparação antes/depois da implementação.
 2. Satisfação: Questionários com garçons, gerentes e clientes.
 3. Tempo médio por pedido: Redução esperada de 30%.
 - Fase 3: Ajustes Pós-Teste
 - Priorizar correções com base em:
 1. Severidade (ex.: erros que bloqueiam operações).
 2. Feedback dos usuários (ex.: melhorias na UI).

9. Boas Práticas de Código na Aplicação (Task 02)

- Single Responsibility Principle (SRP)
 - Princípio da Responsabilidade Única: Uma classe deve ter apenas uma responsabilidade.
 - Nessa aplicação esse princípio pode ser empregado no gerenciamento de pedidos, pois a classe pedido se relaciona com diversas outras classes, sendo essencial separar bem as funções e responsabilidades de cada classe que será implementada para gerenciar os pedidos e suas relações.

Exemplo na aplicação:

- Classe PedidoService:
 - Responsabilidade: Gerenciar a lógica de criação e gerenciamento de pedidos

Errado:

Serviços de pedido não deve ser responsável por realizar pagamento, essa responsabilidade fica para Serviços de pagamento.

```
public class PedidoService {  
    public void realizarPedido() { /* realiza pedido */}  
    public void realizarPagamento() { /* realiza pagamento */}  
}
```

Certo:

Serviços diferentes para realizar pedido e para realizar pagamento.

```
public class PedidoService {  
    public void realizarPedido() { /* realiza pedido */}  
}  
  
public class PagamentoService {  
    public void realizarPagamento() { /* realiza pagamento */}  
}
```

- Open/Closed Principle (OCP)
 - Princípio Aberto/Fechado: O software deve ser aberto para extensão, mas fechado para modificação.
 - Nesta aplicação será necessário cadastrar gateways de pagamento, e para não ter que alterar a classe PagamentoService toda vez que for adicionado um novo gateway, o melhor é criar uma interface de gateway pagamento,

e cada novo tipo criar uma classe com base nessa interface, utilizando o OCP.

Exemplo na aplicação:

- Adição de novos gateways de pagamento

Errado:

```
class PagamentoService {
    void processar(String tipo) {
        if (tipo.equals("MERCADO_PAGO")) {
            // Lógica específica do Mercado Pago
        } else if (tipo.equals("PAGSEGURO")) {
            // Lógica específica do PagSeguro
        }
    }
}
```

Certo:

```
interface GatewayPagamento {
    void processar();
}

class MercadoPagoGateway implements GatewayPagamento {
    void processar() { /* ... */ }
}

class PagSeguroGateway implements GatewayPagamento {
    void processar() { /* ... */ }
}

class PagamentoService {
    void processar(GatewayPagamento gateway) {
        gateway.processar();
    }
}
```

- Liskov Substitution Principle (LSP)
 - Princípio de Substituição de Liskov: Objetos de uma classe derivada devem poder substituir objetos da classe base sem quebrar o comportamento esperado do sistema.

Exemplo na aplicação:

- Permissões dos diferentes tipos de usuários para acessar a dashboard.

Errado:

```
class Usuario {
    public void acessarDashboard() {
        System.out.println(x:"Acessando dashboard genérico ...");
    }
}

class Administrador extends Usuario {
    @Override
    public void acessarDashboard() {
        System.out.println(x:"Acessando dashboard ADMIN ...");
    }
}

class Cliente extends Usuario {
    @Override
    public void acessarDashboard() {
        throw new UnsupportedOperationException(message:"Clientes não têm dashboard!");
    }
}
```

Certo:

```
abstract class Usuario {
    public abstract void acessarRecursoPrincipal();
}

class Administrador extends Usuario {
    @Override
    public void acessarRecursoPrincipal() {
        System.out.println("Acessando painel de administração ...");
    }
}

class Cliente extends Usuario {
    @Override
    public void acessarRecursoPrincipal() {
        System.out.println("Acessando cardápio ...");
    }
}
```

- Interface Segregation Principle (ISP)
 - Princípio da Segregação de Interfaces: As classes não devem ser forçadas a depender de interfaces que contém métodos que não utilizam. Interfaces devem ser específicas para cada contexto.

- Um usuário pode assumir papéis de administrador, cliente e gerente, não é ideal que uma única interface contenha todos os métodos possíveis para todos os tipos de usuários.

Exemplo na aplicação:

- Separação de interfaces para cada tipo de usuário.

Errado:

```
interface UsuarioService {
    public void cadastrarRestaurante();
    public void gerenciarCardapio();
    public void fazerPedido();
}

class Administrador implements UsuarioService {
    public void cadastrarRestaurante() { /* */ }
}

class Gerente implements UsuarioService {
    public void gerenciarCardapio() { /* */ }
}

class Cliente implements UsuarioService {
    public void fazerPedido() { /* */ }
}
```

Certo:

```
interface IAdministradorService {
    public void cadastrarRestaurante();
}

interface IGerenteService {
    public void gerenciarCardapio();
}

interface IClienteService {
    public void fazerPedido();
}

class AdministradorService implements IAdministradorService {
    public void cadastrarRestaurante() { /* */ };
}

class GerenteService implements IGerenteService {
    public void gerenciarCardapio() { /* */ };
}

class ClienteService implements IClienteService {
    public void fazerPedido();
}
```

10. Conclusão

Este sistema de gerenciamento de restaurantes surge como uma solução prática para otimizar o fluxo de pedidos, reduzindo erros e agilizando o atendimento. Com uma arquitetura bem estruturada e foco na experiência do usuário, o MVP valida a viabilidade da proposta antes de escalar para funcionalidades mais complexas. A aplicação dos princípios SOLID garante um código sustentável, enquanto os testes e o piloto em restaurantes reais assegurarão que a solução atenda às necessidades de clientes e estabelecimentos. O próximo passo é colocar os conhecimentos adquiridos em prática e trazer à tona este planejamento.